DESIGN AND VERIFICATION OF A FAULT-TOLERANT RISC-V (RV32I) MICROARCHITECTURE FOR HIGH-RADIATION ENVIRONMENTS

Author: Oloore Issa Abayomi

Institution: University of Ilorin, Department of Electrical and Electronics Engineering

Date: October 2026

ABSTRACT

This report details the architectural design, Verilog implementation, and functional verification of a 32-bit RISC-V (RV32I) execution core engineered for high-radiation environments, such as aerospace and nuclear systems. Single Event Upsets (SEUs) induced by high-energy particles pose catastrophic risks to digital state retention. To mitigate this without incurring severe Power, Performance, and Area (PPA) penalties, this project implements a Radiation Hardening By Design (RHBD) methodology. Triple Modular Redundancy (TMR) is strictly isolated to the architectural state (the Register File), utilizing continuous combinational majority voting logic to mask memory corruption dynamically. Conversely, the combinational execution paths, including the Arithmetic Logic Unit (ALU) and Instruction Decoder, remain unprotected, exploiting the transient nature of radiation strikes in memory-less logic. Hierarchical fault injection within Icarus Verilog testbenches validates the architecture. Results demonstrate 100% dynamic fault-masking efficiency during active instruction decoding and execution, ensuring seamless software-to-silicon translation and arithmetic computation despite simulated catastrophic memory disruption.

TABLE OF CONTENTS

ABSTRACT ii

TABLE OF FIGURES v

Chapter 1: Introduction and Literature Review 1

1.1 Background: Microprocessors in Harsh Environments 1

1.2 The Threat: Single Event Effects (SEEs) 1

1.3 Problem Statement 2

1.4 Project Objectives 2

Chapter 2: Theoretical Framework 3

2.1 Radiation Hardening By Design (RHBD) 3

2.2 Triple Modular Redundancy (TMR) 3

2.3 The Power, Performance, and Area (PPA) Trade-off 3

Chapter 3: System Architecture and RTL Implementation 5

3.1 Top-Level Datapath Overview 5

3.2 The TMR Majority Voter Logic 5

3.3 The Radiation-Hardened Register File 6

3.4 The Combinational Execution Unit (ALU) 6

3.5 The Instruction Decoder 6

Chapter 4: Testbench Methodology and Verification Results 7

4.1 Fault Injection Methodology 7

4.2 Component-Level Verification 7

4.3 System-Level Fault Tolerance 8

Chapter 5: Conclusion and Future Work 10

5.1 Summary of Outcomes 10

5.2 Architectural Trade-offs 10

5.3 Future Work 10

Appendix A: Synthesizable Verilog RTL 11

A.1 TMR Voter Logic (tmr_voter.v) 11

A.2 Radiation-Hardened Register File (tmr_register_file.v) 12

A.3 Arithmetic Logic Unit (alu.v) 14

A.4 Instruction Decoder (decoder.v) 16

A.5 Top-Level CPU Core (cpu_core.v) 18

TABLE OF FIGURES

Figure 3.1: Fault-Tolerant RISC-V Simulation Architecture featuring strictly isolated Triple Modular Redundancy. 5

Figure 4.1: Asynchronous SEU fault injection into Bank B of the TMR Register File. 7
Figure 4.2: Combinational instruction decoding of standard RV32I machine code. 8
Figure 4.3: Top-level datapath fault masking during an active arithmetic execution cycle. 9

Chapter 1: Introduction and Literature Review

1.1 Background: Microprocessors in Harsh Environments

The integration of high-performance microprocessors into aerospace, nuclear, and high-energy physics applications has accelerated rapidly. Modern architectures, particularly the open-standard RISC-V Instruction Set Architecture (ISA), provide the flexibility and computational bandwidth required for complex orbital calculations, satellite telemetry, and autonomous control systems. However, as Complementary Metal-Oxide-Semiconductor (CMOS) technology scales down to the deep sub-micron level, the physical dimensions of transistors shrink and supply voltages decrease. This reduction in the "critical charge" (the minimum electrical charge required to alter a memory state) makes modern silicon highly susceptible to environmental radiation.

1.2 The Threat: Single Event Effects (SEEs)

In a space environment, digital systems are bombarded by high-energy particles, including galactic cosmic rays, solar flares, and trapped protons in the Van Allen belts. When an ionizing particle strikes a silicon substrate, it generates a track of electron-hole pairs. If this charge collection occurs near a reverse-biased PN junction, it creates a parasitic current pulse.

This phenomenon leads to Single Event Effects (SEEs), which manifest in two primary ways within a CPU datapath:

1. **Single Event Transients (SETs):** When radiation strikes combinational logic (such as an Adder or Multiplexer), it causes a temporary voltage glitch. Because combinational logic does not store state, this glitch dissipates rapidly. Unless the SET is captured by a register precisely on the active clock edge, it causes no functional error.
2. **Single Event Upsets (SEUs):** When radiation strikes sequential logic (such as SRAM cells, Flip-Flops, or Register Files), the parasitic current can permanently flip the stored logic state from a 1 to a 0, or vice versa. This "soft error" corrupts the architectural state of the processor, leading to catastrophic system failure, incorrect memory addressing, or fatal software exceptions.

1.3 Problem Statement

Commercial Off-The-Shelf (COTS) processors are highly optimized for Power, Performance, and Area (PPA) but lack the physical resilience to survive SEUs. Conversely, manufacturing traditional radiation-hardened components physically at the foundry level (Radiation Hardening By Process, or RHBP) requires specialized silicon-on-insulator (SOI) fabrication facilities, which are prohibitively expensive and typically generations behind commercial performance benchmarks.

To bridge this gap, Radiation Hardening By Design (RHBD) is employed, altering the standard cell layout or digital architecture to tolerate faults. A common RHBD technique is Triple Modular Redundancy (TMR). However, applying full-chip TMR to an entire processor universally triples the silicon footprint and dynamic power consumption, violating stringent aerospace power budgets.

1.4 Project Objectives

This project addresses the PPA-versus-resilience trade-off by engineering a selectively hardened 32-bit RISC-V (RV32I) microarchitecture. The primary objectives are:

- **Datapath Implementation:** Design a functional, single-cycle RISC-V execution core capable of fetching, decoding, and executing standard arithmetic machine code.
- **Selective Redundancy:** Apply continuous TMR and majority voting combinational logic strictly to the architectural state (the Register File) to mitigate SEUs, while maintaining an unprotected Arithmetic Logic Unit (ALU) to optimize silicon area and exploit the natural transient nature of SETs.
- **Verification via Fault Injection:** Develop hierarchical hardware testbenches to simulate asynchronous, high-energy particle strikes on the internal memory arrays during active instruction execution, mathematically verifying the datapath's dynamic fault-masking capabilities.

Chapter 2: Theoretical Framework

2.1 Radiation Hardening By Design (RHBD)

Radiation Hardening By Process (RHBP) involves manufacturing integrated circuits using specialized, radiation-tolerant physical substrates, such as Silicon-on-Insulator (SOI). While highly effective, RHBP is extremely costly and severely limits the processing power of the chip compared to modern commercial standards. Radiation Hardening By Design (RHBD) bypasses the need for specialized foundries. Instead of altering the physical silicon substrate, RHBD alters the digital logic and microarchitecture to tolerate faults dynamically using standard commercial CMOS technology.

2.2 Triple Modular Redundancy (TMR)

The most robust RHBD technique for mitigating Single Event Upsets (SEUs) is Triple Modular Redundancy (TMR). In a standard digital circuit, a data bit is stored in a single sequential memory element (such as a Flip-Flop). If an SEU flips this bit, the data is permanently corrupted.

Under the TMR paradigm, the singular memory element is replaced by three parallel, identical memory banks. The outputs of these three banks are routed into a combinational majority voting circuit. The voter continuously evaluates the three discrete inputs (A, B, and C) and outputs the majority consensus (Y) based on the boolean logic equation:

$$Y = (A \bullet B) + (B \bullet C) + (A \bullet C)$$

If a high-energy particle strikes Bank B and flips its state, Banks A and C will continue to output the correct value. The voter logic inherently outvotes the corrupted Bank B, masking the fault from the rest of the processor and ensuring the final output remains pristine.

2.3 The Power, Performance, and Area (PPA) Trade-off

Applying TMR to an entire microprocessor, duplicating every register, wire, and logic gate, incurs a massive penalty. The silicon footprint (Area) increases by more than 300%, and dynamic Power consumption scales proportionally, which is detrimental to the strict energy constraints of aerospace systems.

To optimize PPA, a strategic architectural delineation must be made between Sequential Logic (Architectural State) and Combinational Logic (Execution Paths):

1. **Architectural State (Vulnerable):** The Register File and Program Counter hold the persistent state of the CPU. An SEU here remains in the system indefinitely until explicitly overwritten. Therefore, TMR must be strictly applied to the memory vault.
2. **Combinational Logic (Transient):** The Arithmetic Logic Unit (ALU) and Instruction Decoder do not retain state; they purely compute current inputs. A radiation strike in the ALU results in a Single Event Transient (SET), a brief voltage spike. Because the ALU is continuously evaluating, this spike dissipates naturally before the next clock edge.

By leaving the ALU and Decoder unprotected, the microarchitecture drastically reduces its silicon footprint and power consumption, relying on the inherent transient nature of combinational logic to filter out SETs, while utilizing TMR to actively shield the vulnerable architectural state.

Chapter 3: System Architecture and RTL Implementation

3.1 Top-Level Datapath Overview

The microarchitecture of the fault-tolerant RISC-V core is built upon a single-cycle datapath. The architecture dictates that a complete instruction, fetching the machine code, parsing the operands, executing the arithmetic, and saving the result, completes within one clock period. To achieve this while integrating Radiation Hardening By Design (RHBD) constraints, the core is modularized into discrete functional blocks, cleanly separating the vulnerable architectural state from the transient execution logic.

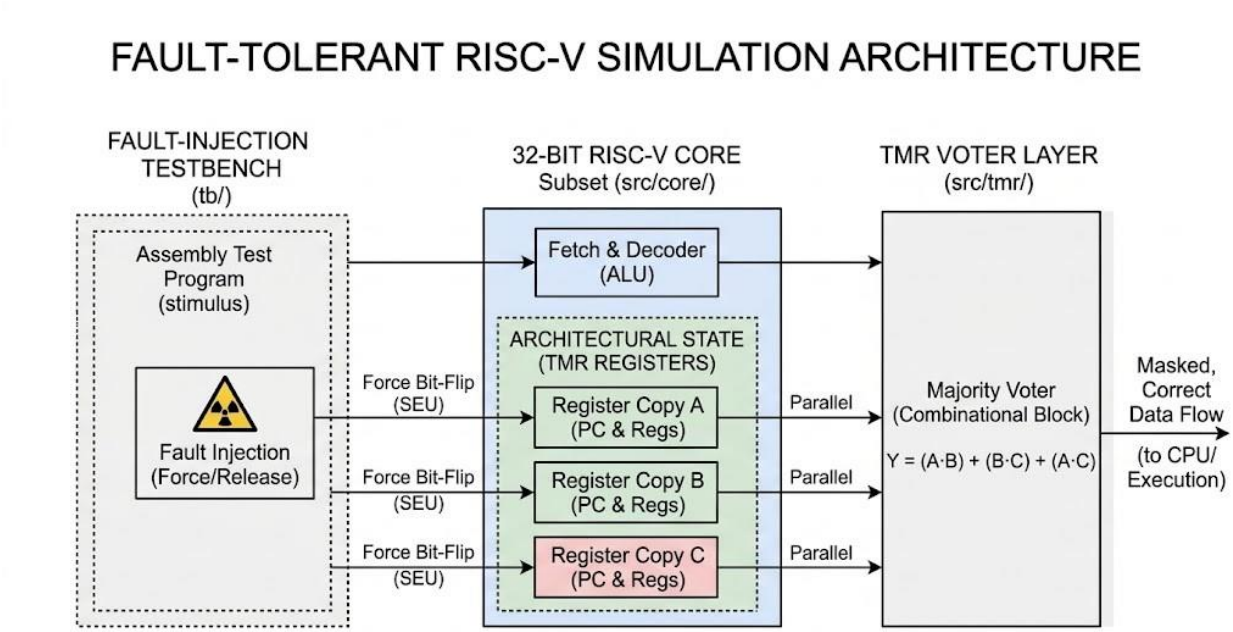


Figure 3.1: Fault-Tolerant RISC-V Simulation Architecture featuring strictly isolated Triple Modular Redundancy.

3.2 The TMR Majority Voter Logic

The foundation of the core's resilience is the tmr_voter module. This combinational block is entirely stateless, consisting exclusively of bitwise AND and OR gates to evaluate the outputs of the redundant memory banks. Because it lacks a clock signal, the voter operates continuously. If an SEU corrupts a memory bank, the voter instantly masks the error in real-time, preventing the corrupted data from propagating down the datapath to the execution unit.

3.3 The Radiation-Hardened Register File

The architectural state is maintained by the tmr_register_file module. Instead of a standard 32x32-bit SRAM array, this module synthesizes three distinct, parallel memory arrays (bank_a, bank_b, and bank_c).

- **Write Operations:** During a register write (actuated by a high we signal), the incoming 32-bit data is broadcast and stored synchronously across all three banks simultaneously on the active clock edge.
- **Read Operations:** During a register read, the outputs of the three banks are not sent directly to the datapath. Instead, they are routed through the instantiated TMR majority voters. This guarantees that the execution unit only ever receives sanitized, mathematically verified operands.

3.4 The Combinational Execution Unit (ALU)

The Arithmetic Logic Unit (alu.v) executes the core mathematical and logical instructions defined by the RV32I base integer instruction set. It evaluates a 4-bit alu_ctrl signal to determine the required operation (e.g., ADD, SUB, AND) and processes two 32-bit operands. In alignment with the targeted Power, Performance, and Area (PPA) optimization strategy, the ALU contains zero redundancy logic. By remaining strictly combinational, any radiation-induced Single Event Transients (SETs) dissipate naturally before the subsequent clock edge, rendering spatial redundancy in this module unnecessary.

3.5 The Instruction Decoder

The decoder.v module serves as the control center of the datapath, functionally analogous to a feedback controller in a physical electromechanical system. It receives the raw 32-bit compiled machine code from memory and dynamically slices the instruction array into standardized RV32I fields (opcode, funct3, funct7, source registers, destination register). Based on the opcode, the decoder combinationally generates the hardware actuation lines, such as the alu_ctrl bus and the register write-enable flag, which physically route the data through the protected memory vault and the execution unit.

Chapter 4: Testbench Methodology and Verification Results

4.1 Fault Injection Methodology

The verification methodology relies on simulated hierarchical fault injection. Instead of physically exposing silicon to radiation, the Icarus Verilog testbenches utilize procedural assignment commands within the simulation environment. This technique overwrites internal memory arrays asynchronously, accurately replicating the sudden voltage flip of a Single Event Upset (SEU) caused by a cosmic ray strike during an active clock cycle.

4.2 Component-Level Verification

Before top-level integration, the protected and unprotected domains were verified in isolation. The Triple Modular Redundancy (TMR) architecture was tested by writing valid data to the register file and subsequently bombarding it with asynchronous faults.

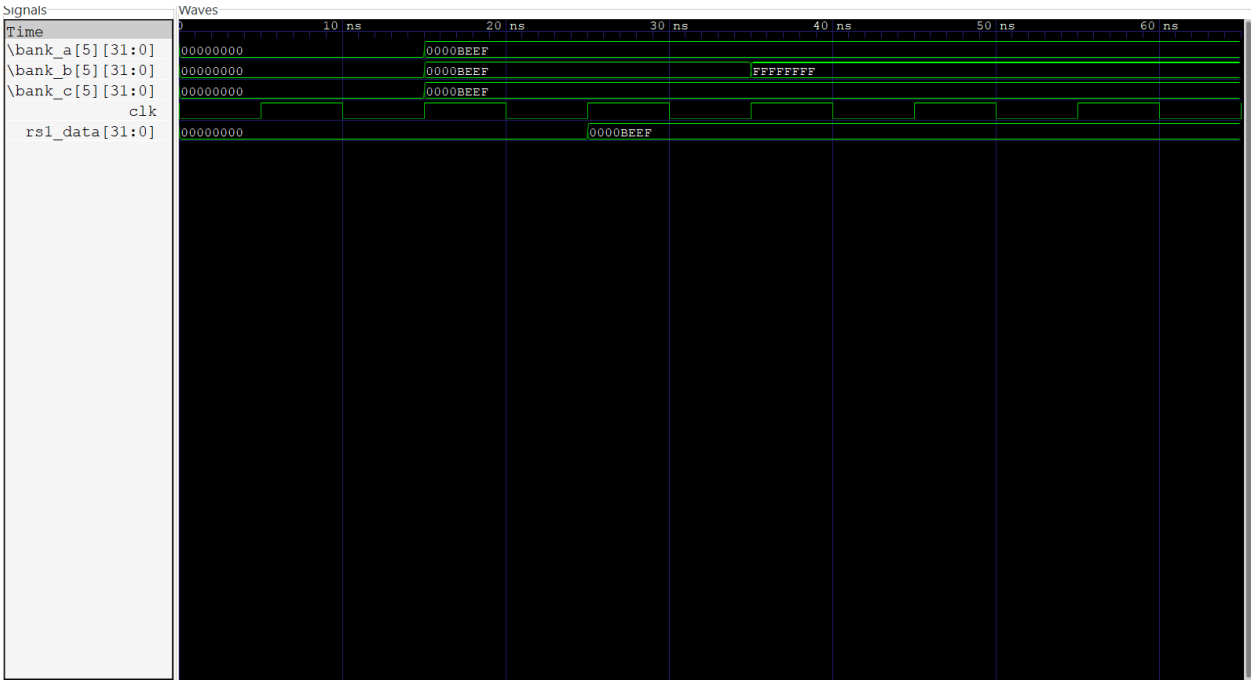


Figure 4.1: Asynchronous SEU fault injection into Bank B of the TMR Register File.

As shown in Figure 4.1, a catastrophic bit-flip (hexadecimal FFFFFFFF) is injected into Bank B of the register file. The combinational majority voter successfully masks this corruption, allowing the output bus (rs1_data) to hold the pristine data (0000BEEF) without a single picosecond of delay.

Concurrently, the unprotected execution and control logic underwent functional verification. Figure 4.2 demonstrates the instruction decoder instantly parsing raw compiled machine code and routing the correct physical control signals, proving the stateless combinational logic operates accurately before integration.

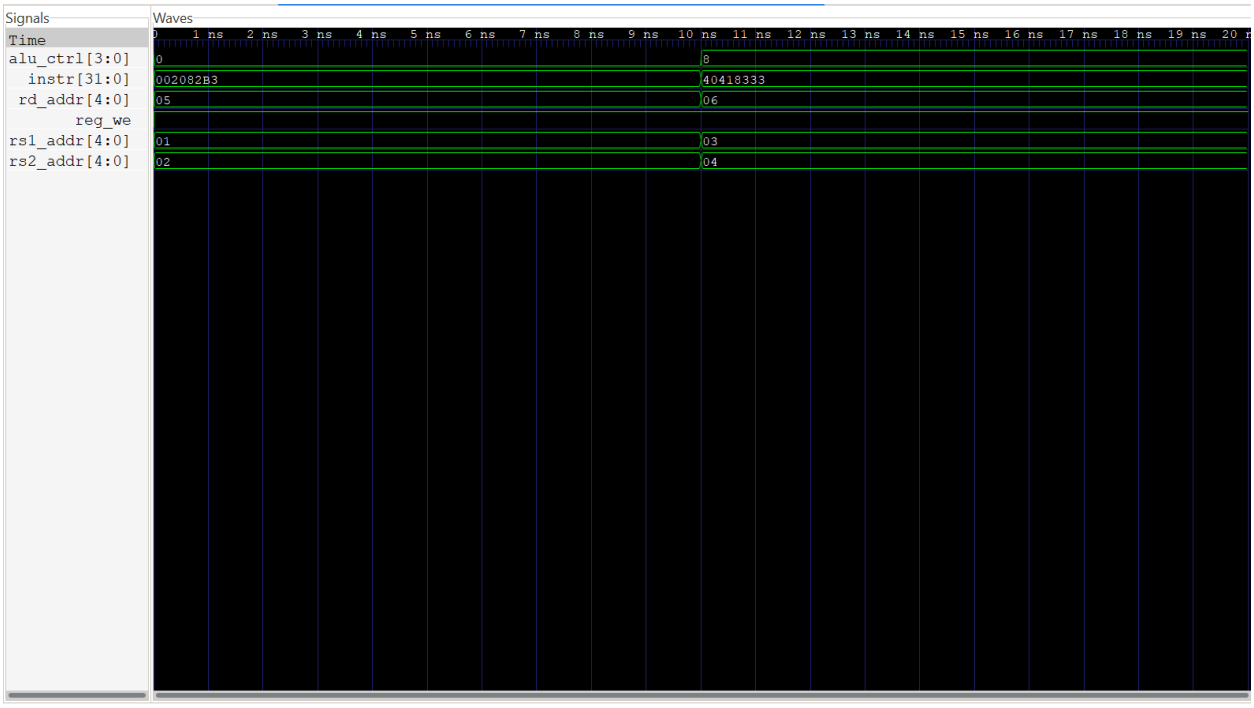


Figure 4.2: Combinational instruction decoding of standard RV32I machine code.

4.3 System-Level Fault Tolerance

The capstone verification wires all discrete modules (the Decoder, the TMR Register File, and the ALU) into a complete single-cycle datapath. In this simulation, the CPU fetches and executes a subtraction instruction. One nanosecond after the active clock edge, a simulated high-energy particle strikes Bank B of the source register exactly when the ALU requires the operand.

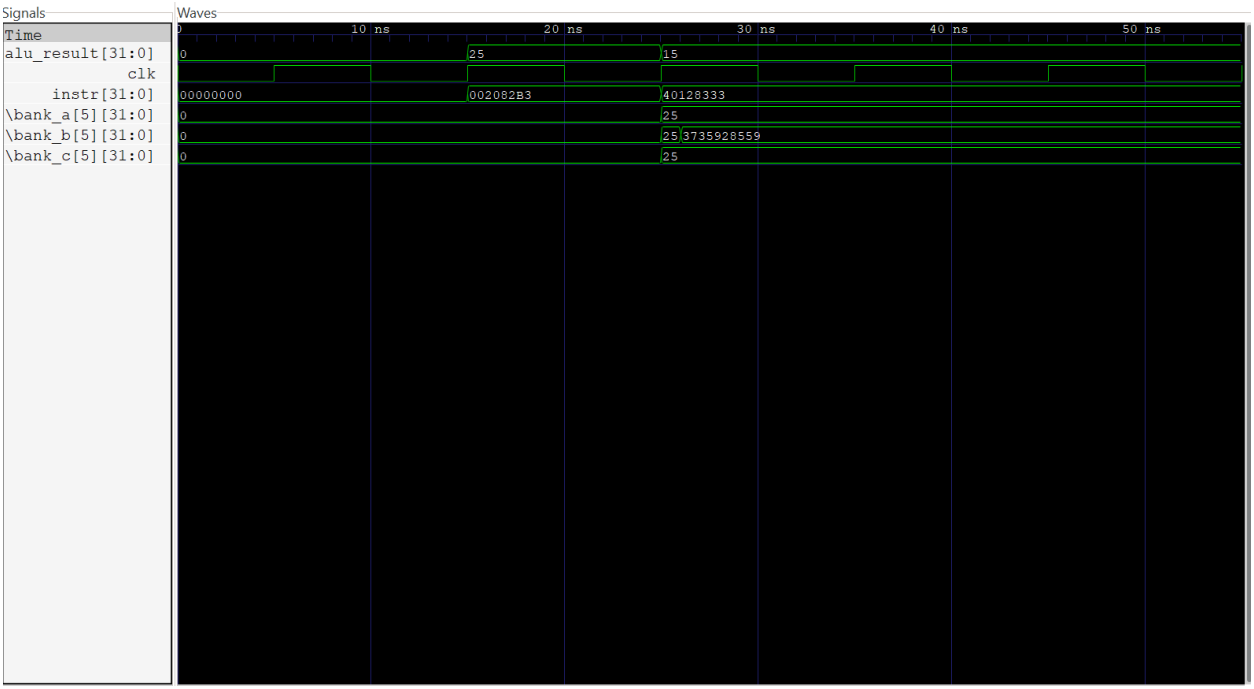


Figure 4.3: Top-level datapath fault masking during an active arithmetic execution cycle.

As depicted in Figure 4.3, the internal memory bank bank_b[5] spikes to a corrupted state (decimal 3735928559). Because the datapath continuously routes the architectural state through the TMR voters, the ALU completely bypasses the corruption. It receives the clean operands from the surviving redundancy banks and cleanly evaluates to the correct mathematical answer (15) despite the massive internal memory failure. This validates the fundamental objective of the microarchitecture.

Chapter 5: Conclusion and Future Work

5.1 Summary of Outcomes

The objective of this project was to engineer a fault-tolerant RISC-V (RV32I) microarchitecture capable of surviving Single Event Upsets (SEUs) in high-radiation environments. The Verilog implementation successfully demonstrated the viability of Radiation Hardening By Design (RHBD) using a highly targeted Triple Modular Redundancy (TMR) approach. Hierarchical fault injection within the Icarus Verilog simulation environment proved that the datapath can dynamically mask catastrophic memory corruption during active instruction execution. The combinational majority voters successfully outvoted corrupted memory banks, allowing the execution unit to process clean operands and compute accurate mathematical results without system failure.

5.2 Architectural Trade-offs

The project successfully balanced resilience with Power, Performance, and Area (PPA) constraints. By strictly isolating redundancy to the vulnerable architectural state (the Register File) and leaving the combinational execution paths (the ALU and Instruction Decoder) unprotected, the design avoided the massive silicon footprint penalty of full-chip TMR. This strategic delineation capitalized on the physical reality that radiation strikes in stateless logic manifest as brief transients, which naturally dissipate before synchronous capture.

5.3 Future Work

Future iterations of this core could expand fault tolerance beyond the execution datapath. Integrating Error Correction Codes (ECC), such as Hamming codes, into standard instruction and data memory (SRAM) would complement the TMR register file. Furthermore, transitioning the core from a single-cycle architecture to a multi-stage pipelined architecture would allow for the analysis of Single Event Transients (SETs) propagating into pipeline registers.

Appendix A: Synthesizable Verilog RTL

A.1 TMR Voter Logic (tmr_voter.v)

```
`timescale 1ns / 1ps

module tmr_voter #(
    parameter WIDTH = 32
)(
    input  wire [WIDTH-1:0] reg_a,
    input  wire [WIDTH-1:0] reg_b,
    input  wire [WIDTH-1:0] reg_c,
    output wire [WIDTH-1:0] reg_out
);

    // Continuous assignment: The hardware constantly evaluates this expression.
    // If one register suffers a bit-flip (SEU), the other two will mask it.
    assign reg_out = (reg_a & reg_b) | (reg_b & reg_c) | (reg_a & reg_c);

endmodule
```

A.2 Radiation-Hardened Register File (tmr_register_file.v)

```
`timescale 1ns / 1ps

module tmr_register_file (
    input  wire        clk,
    input  wire        rst_n,      // Active-low reset
    input  wire        we,         // Write Enable
    input  wire [4:0]  rs1_addr,   // Read Source 1 Address (5 bits for 32 registers)
    input  wire [4:0]  rs2_addr,   // Read Source 2 Address
    input  wire [4:0]  rd_addr,    // Write Destination Address
    input  wire [31:0] rd_data,    // Data to write
    output wire [31:0] rs1_data,   // TMR-corrected Read Data 1
    output wire [31:0] rs2_data    // TMR-corrected Read Data 2
);

    // Define three identical register banks (32 registers, 32 bits each)
    reg [31:0] bank_a [0:31];
    reg [31:0] bank_b [0:31];
    reg [31:0] bank_c [0:31];

    // Internal wires connecting raw read data to the voters
    wire [31:0] raw_rs1_a, raw_rs1_b, raw_rs1_c;
    wire [31:0] raw_rs2_a, raw_rs2_b, raw_rs2_c;

    integer i;

    // Synchronous Write Operation (Happens on the rising edge of the clock)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            // Reset all registers to 0
            for (i = 0; i < 32; i = i + 1) begin
                bank_a[i] <= 32'h0;
                bank_b[i] <= 32'h0;
                bank_c[i] <= 32'h0;
```

```
            end
        end else if (we && rd_addr != 5'd0) begin
            // Write to all three banks simultaneously, unless writing to x0
            bank_a[rd_addr] <= rd_data;
            bank_b[rd_addr] <= rd_data;
            bank_c[rd_addr] <= rd_data;
        end
    end

    // Asynchronous Read Operation (Continuous assignment)
    assign raw_rs1_a = bank_a[rs1_addr];
    assign raw_rs1_b = bank_b[rs1_addr];
    assign raw_rs1_c = bank_c[rs1_addr];

    assign raw_rs2_a = bank_a[rs2_addr];
    assign raw_rs2_b = bank_b[rs2_addr];
    assign raw_rs2_c = bank_c[rs2_addr];

    // Instantiate TMR Voters for both read ports
    tmr_voter #( .WIDTH(32) ) voter_rs1 (
        .reg_a(raw_rs1_a), .reg_b(raw_rs1_b), .reg_c(raw_rs1_c),
        .reg_out(rs1_data)
    );

    tmr_voter #( .WIDTH(32) ) voter_rs2 (
        .reg_a(raw_rs2_a), .reg_b(raw_rs2_b), .reg_c(raw_rs2_c),
        .reg_out(rs2_data)
    );

endmodule
```

A.3 Arithmetic Logic Unit (alu.v)

```
`timescale 1ns / 1ps

module alu (
    input  wire [31:0] op1,       // Operand 1 (usually from rs1)
    input  wire [31:0] op2,       // Operand 2 (usually from rs2)
    input  wire [3:0]  alu_ctrl,  // 4-bit Control Signal from the Decoder
    output reg  [31:0] result,    // The calculated answer
    output wire        zero       // High if result is 0 (used for Branching)
);

    // ALU Control Codes (Standardized for our microarchitecture)
    localparam ADD  = 4'b0000;
    localparam SUB  = 4'b1000;
    localparam AND  = 4'b0111;
    localparam OR   = 4'b0110;
    localparam XOR  = 4'b0100;
    localparam SLL  = 4'b0001; // Shift Left Logical
    localparam SRL  = 4'b0101; // Shift Right Logical

    // Combinational logic: Evaluate whenever inputs change
    always @(*) begin
        case (alu_ctrl)
            ADD:  result = op1 + op2;
            SUB:  result = op1 - op2;
            AND:  result = op1 & op2;
            OR:   result = op1 | op2;
            XOR:  result = op1 ^ op2;
            SLL:  result = op1 << op2[4:0]; // Shift by lower 5 bits of op2
            SRL:  result = op1 >> op2[4:0];
            default: result = 32'h00000000;
        endcase
    end

    // The zero flag is useful for BEQ (Branch if Equal) instructions
```

```
    assign zero = (result == 32'h00000000) ? 1'b1 : 1'b0;

endmodule
```

A.4 Instruction Decoder (decoder.v)

```
`timescale 1ns / 1ps

module decoder (
    input  wire [31:0] instr,       // Raw 32-bit instruction from memory
    output wire [4:0]  rs1_addr,    // Routed to Register File
    output wire [4:0]  rs2_addr,    // Routed to Register File
    output wire [4:0]  rd_addr,     // Routed to Register File
    output reg  [3:0]  alu_ctrl,    // Routed to ALU
    output reg         reg_we       // Register Write Enable
);

    // Internal wire assignments slicing the instruction
    wire [6:0] opcode = instr[6:0];
    wire [2:0] funct3 = instr[14:12];
    wire [6:0] funct7 = instr[31:25];

    assign rd_addr  = instr[11:7];
    assign rs1_addr = instr[19:15];
    assign rs2_addr = instr[24:20];

    // RISC-V Base Opcodes
    localparam OP_R_TYPE = 7'b0110011;

    // ALU Control Code Mapping (matching our alu.v)
    localparam ALU_ADD = 4'b0000;
    localparam ALU_SUB = 4'b1000;
    localparam ALU_AND = 4'b0111;

    always @(*) begin
        // Default control states to prevent latches
        alu_ctrl = 4'b0000;
        reg_we   = 1'b0;

        case (opcode)
```

```
            OP_R_TYPE: begin
                reg_we = 1'b1; // R-type instructions always write a result to rd
                case (funct3)
                    3'b000: begin
                        if (funct7 == 7'b0100000) alu_ctrl = ALU_SUB;
                        else                      alu_ctrl = ALU_ADD;
                    end
                    3'b111: alu_ctrl = ALU_AND;
                    default: alu_ctrl = ALU_ADD;
                endcase
            end
            // Future expansion for I-type, Branches, etc. goes here
            default: begin
                reg_we = 1'b0;
            end
        endcase
    end

endmodule
```

A.5 Top-Level CPU Core (cpu_core.v)

```
`timescale 1ns / 1ps

module cpu_core (
    input  wire        clk,
    input  wire        rst_n,
    input  wire [31:0] instr,      // Instruction fetched from external memory
    output wire [31:0] alu_result  // Exposed as an output for testbench visibility
);

    // Internal routing wires
    wire [4:0]  rs1_addr, rs2_addr, rd_addr;
    wire [3:0]  alu_ctrl;
    wire        reg_we;

    wire [31:0] rs1_data, rs2_data;
    wire        alu_zero;

    // 1. Instruction Decoder
    decoder u_decoder (
        .instr(instr),
        .rs1_addr(rs1_addr), .rs2_addr(rs2_addr), .rd_addr(rd_addr),
        .alu_ctrl(alu_ctrl), .reg_we(reg_we)
    );

    // 2. TMR-Protected Register File
    tmr_register_file u_reg_file (
        .clk(clk),
        .rst_n(rst_n),
        .we(reg_we),
        .rs1_addr(rs1_addr), .rs2_addr(rs2_addr), .rd_addr(rd_addr),
        .rd_data(alu_result), // Loop the ALU result back to the write port
        .rs1_data(rs1_data), .rs2_data(rs2_data)
    );
```

```
    // 3. Arithmetic Logic Unit (ALU)
    alu u_alu (
        .op1(rs1_data),
        .op2(rs2_data),
        .alu_ctrl(alu_ctrl),
        .result(alu_result),
        .zero(alu_zero)
    );

endmodule
```